

설문 데이터 분류 체계: Taxonomy and Ontology 시스템

작성일: 2025-11-05 버전: 1.1 대상: AI Lab 최종 수정: 2025-11-05 - 온톨로지 관계 기반 척도 정의로 개정

Executive Summary

핵심 문제

설문 조사 문항의 분류 체계는 고정된 계층 구조로는 관리할 수 없을 정도로 복잡하고 동적입니다. 조직진단 (OD)만 해도 대분류 10-15개, 각각의 중분류 3-10개, 소분류 2-5개로 수백~수천 가지 조합이 가능하며, 매년 새로운 개념(ESG, 디지털전환, 심리적 안전 등)이 등장하고 고객사마다 특화된 분류를 요구합니다.

제안 솔루션

하이브리드 분류 체계를 도입합니다:

- 명확한 메타데이터** (별도 컬럼): 진단 유형(OD/LD/ES), 연도, 회사, 업종 등 변하지 않는 분류 기준
- 태그 기반 온톨로지 (JSONB)**: 조직문화, 리더십, 비전 등 복잡하고 진화하는 의미론적 분류

이를 통해 명확한 구조적 분류와 유연한 의미론적 분류를 동시에 수용하며, 무한 확장이 가능하고 시간이 지날 수록 학습하며 진화하는 분류 체계를 구축합니다.

핵심 이점

특성	기존 방식	제안 방식
확장성	새 분류 추가시 스키마 변경 필요	데이터만 추가, 구조 변경 불필요
유연성	고정된 3-4단계 계층만 가능	2-10단계 모두 수용 가능
검색	정확히 일치하는 것만 검색	의미 기반 관련 항목 자동 검색
학습	수동으로만 관리	자동 학습 및 제안
복잡도	복잡도 증가시 관리 불가능	복잡도와 무관하게 일관된 관리

구현 시간

- 최소 구현**: 1-2주 (핵심 기능만)
- 완전 구현**: 4-6주 (자동화 포함)
- ROI 시점**: 3-6개월 (신규 분류 추가시마다 절감 효과 증가)

1. 문제 배경 (Problem Context)

1.1 설문 분류의 본질적 복잡성

실제 분류 체계의 규모

조직진단 (Organizational Diagnosis)

└ 대분류 (Level 1): 10-15개

- └ 비전/미션
- └ 조직문화
- └ 리더십
- └ 소통/커뮤니케이션
- └ 협력/팀워크
- └ 혁신
- └ 변화관리
- └ 성과관리
- └ 인재육성
- └ 일생활균형 (워라밸)
- └ 다양성/포용성 (D&I)
- └ 디지털전환 (DX)
- └ ESG/지속가능경영
- └ 고객지향성
- └ 안전/보건

└ 중분류 (Level 2): 각 대분류당 3-10개

- 예) 리더십
- └ 변혁적 리더십
 - └ 서번트 리더십
 - └ 코칭
 - └ 피드백
 - └ 의사결정
 - └ 권한위임
 - └ 비전 제시
 - └ 인재육성

└ 소분류 (Level 3): 각 중분류당 2-5개

- 예) 코칭
- └ 정기성
 - └ 질적 수준
 - └ 실행력 향상
 - └ 성장 지원

총 조합 가능성: $10 \times 5 \times 3 = 150$ 개 이상 (단순 계산)

실제: 의미론적 조합까지 고려하면 **500-1,000개 이상**

다른 진단 유형

- 리더십 진단 (LD): 100-200개 분류
- 몰입도 진단 (ES): 80-150개 분류
- 다면평가 (MA): 50-100개 분류
- 고객사 커스텀: 회사당 10-50개 추가

전체 분류 항목: 최소 800개 ~ 최대 2,000개 이상

1.2 시간적 변화 (Temporal Evolution)

과거 10년간의 분류 진화

2015년:

- 전통적 조직 요소 중심
- 리더십, 소통, 협력, 성과관리
- 약 200개 분류 항목

2018년:

- 디지털 트랜스포메이션 추가
- 애자일, 유연근무, 세대 갈등
- 약 350개 분류 항목

2020년 (팬데믹):

- 원격근무, 디지털 협업
- 심리적 안전, 회복탄력성
- 약 500개 분류 항목

2023년:

- ESG, 지속가능경영
- DEI (다양성, 형평성, 포용성)
- AI/자동화 적응력
- 약 700개 분류 항목

2025년 (현재):

- 생성형 AI 활용도
- 하이브리드 워크 최적화
- 직원 경험 (Employee Experience)
- **예상: 1,000개 이상**

미래 예측

- **2027년:** 메타버스 협업, Web3 조직 구조 → +200개
- **2030년:** AI 에이전트 협업, 자율 조직 → +300개

결론: 분류 체계는 정적(static)이 아니라 동적(dynamic)이며 지속적으로 진화

1.3 고객사별 특수성

회사별 커스텀 요구사항

CJ그룹:

- "ONLYONE" (핵심가치)
 - "건강한 식문화" (비전)
 - "문화창조기업" (아이덴티티)
- 10-15개 CJ 전용 분류 필요

삼성전자:

- "초격차 기술"
- "반도체 경쟁력"

- "글로벌 시민"
- 15-20개 삼성 전용 분류 필요

제조업 vs IT업:

- 제조: 안전, 품질, 생산성 강조
- IT: 혁신, 속도, 창의성 강조
- 업종별 특화 분류 50-100개

문제점

고정된 계층 구조로는 이러한 **다양성과 특수성을 동시에 수용할 수 없음**

1.4 의미론적 모호성 (Semantic Ambiguity)

같은 개념, 다른 표현

"소통" = "커뮤니케이션" = "의사소통" = "정보공유"

→ 같은가? 다른가? 어떻게 분류?

"워라밸" = "일생활균형" = "Work-Life Balance"

→ 공식 용어 변경시 과거 데이터는?

"리더십" vs "매니지먼트" vs "관리역량"

→ 미묘한 차이를 어떻게 구분?

계층적 모호성

"협력"은 대분류인가 중분류인가?

- A회사: 대분류 (중요도 높음)
- B회사: 중분류 (조직문화 > 협력)

"디지털전환"은 어디에 속하는가?

- 전략 > 디지털전환
- 혁신 > 디지털전환
- 기술역량 > 디지털전환
- 모두 타당함!

결론: 분류는 계층적이지 않고 네트워크적(networked)

2. 현황 분석: 기존 접근법의 한계

2.1 고정 계층 구조 방식 (Fixed Hierarchy)

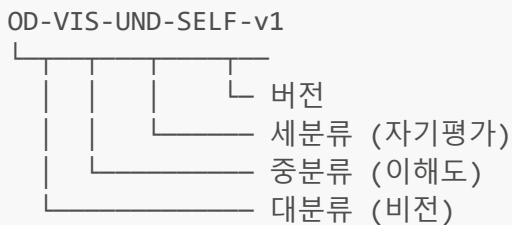
현재 스키마

```
CREATE TABLE questions (
  question_id VARCHAR(50) PRIMARY KEY,
  question_text TEXT,

  -- 고정된 3-4단계 계층
  category_1 VARCHAR(50), -- 대분류
  category_2 VARCHAR(50), -- 중분류
  category_3 VARCHAR(50), -- 소분류
  category_4 VARCHAR(50), -- 세분류

  diagnosis_type VARCHAR(20), -- OD, LD, ES
  scale_type VARCHAR(20)
);
```

문항 코드 예시



2.2 한계점 상세 분석

한계 1: 확장성 부족

```
-- 문제 상황: 5단계 분류가 필요한 경우
-- 예: 조직문화 > 협력 > 팀워크 > 심리적안전 > 발언권

-- ✕ 불가능: category_5 컬럼이 없음
INSERT INTO questions VALUES (
  'OD-CUL-COL-TW-PS-VOICE-v1', -- 6단계 코드
  ...,
  '조직문화', -- category_1
  '협력', -- category_2
  '팀워크', -- category_3
  '심리적안전' -- category_4
  -- ✕ '발언권'을 어디에? category_5 없음!
);

-- 해결책: 스키마 변경 (ALTER TABLE)
-- 문제: 기존 데이터 마이그레이션, 애플리케이션 수정 필요
```

한계 2: 계층 유연성 부족

```
-- A회사: 3단계 분류
category_1: '리더십'
category_2: '코칭'
category_3: '정기성'
category_4: NULL

-- B회사: 2단계 분류
category_1: '리더십'
category_2: '코칭'
category_3: NULL
category_4: NULL

-- C회사: 5단계 분류
category_1: '조직문화'
category_2: '협력'
category_3: '팀워크'
category_4: '심리적안전'
-- ✕ 5단계는 불가능!

-- 문제: 모든 회사가 같은 계층 깊이를 강요당함
```

한계 3: 다차원 분류 불가능

```
-- 현실: 문항은 여러 차원에 동시 속함
"리더가 팀원의 디지털 전환 역량을 코칭하고 있습니까?"

-- 의미상 관련된 모든 분류:
- 리더십 > 코칭
- 디지털전환 > 역량개발
- 인재육성 > 스킬개발
- 변화관리 > 적응력

-- ✕ 고정 계층은 하나만 선택 가능
category_1: '리더십' -- 또는 '디지털전환'? 또는 '인재육성'?
category_2: '코칭'
category_3: '역량개발'

-- 나머지 의미들은 손실됨!
```

한계 4: 검색 한계

```
-- 사용자 요구: "리더십 관련 모든 문항"
SELECT * FROM questions
WHERE category_1 = '리더십';

-- ✕ 놓치는 것들:
-- - category_2가 '리더십'인 것들
-- - '코칭', '피드백', '권한위임' 등 리더십 하위 개념들
```

```
-- - '변혁적 리더십', '서번트 리더십' 등 관련 개념들

-- 현실적 쿼리 (복잡하고 불완전):
SELECT * FROM questions
WHERE category_1 = '리더십'
   OR category_2 = '리더십'
   OR category_1 IN ('코칭', '피드백', '권한위임', ...)
   OR question_text LIKE '%리더%'
   OR question_text LIKE '%리더십%';

-- 수동 관리 필요, 항상 불완전
```

한계 5: 신규 개념 추가 어려움

```
-- 2025년: "생성형 AI 활용도" 개념 등장

-- 기존 분류 체계에 어디에 넣을 것인가?
-- 옵션 1: 디지털전환 > AI활용
-- 옵션 2: 혁신 > 기술수용
-- 옵션 3: 업무효율 > 도구활용
-- 옵션 4: 새로운 대분류 추가?

-- 문제:
-- 1. 정답이 없음 (회사마다 다를 수 있음)
-- 2. 한번 정하면 변경 어려움
-- 3. 과거 분류와 일관성 유지 어려움
```

한계 6: 동의어/유사어 관리 불가

```
-- "소통", "커뮤니케이션", "의사소통"을 같은 것으로 검색하려면?

-- ✕ 현재 방식: 모두 별도로 검색
SELECT * FROM questions WHERE category_2 = '소통'
UNION
SELECT * FROM questions WHERE category_2 = '커뮤니케이션'
UNION
SELECT * FROM questions WHERE category_2 = '의사소통';

-- 문제: 동의어 목록을 수동으로 관리해야 함
```

2.3 실제 운영 시나리오에서의 문제

시나리오 1: 연도별 비교 분석

```
-- 요구사항: "2023년과 2024년 '비전' 관련 점수 비교"

-- 문제:
```

```
-- 2023년: category_1 = '비전'
-- 2024년: category_1 = '전략', category_2 = '비전' (재분류됨)

-- ✕ 단순 비교 불가능
SELECT year, AVG(score)
FROM responses r
JOIN questions q ON r.question_id = q.question_id
WHERE category_1 = '비전' -- 2024년 데이터 누락!
GROUP BY year;

-- 해결: 수동 매핑 테이블 유지보수 필요 (노동집약적)
```

시나리오 2: 고객사 커스텀 분류

```
-- CJ: "ONLYONE" 핵심가치 관련 문항

-- 옵션 1: 기존 분류에 억지로 끼워넣기
category_1: '조직문화'
category_2: 'ONLYONE' -- ✕ 다른 회사에는 의미 없는 분류

-- 옵션 2: 별도 테이블
CREATE TABLE cj_custom_categories (...); -- ✕ 통합 분석 어려움

-- 옵션 3: category_4에 회사코드
category_4: 'CJ-ONLYONE' -- ✕ 세분류와 회사코드가 혼재

-- 모든 옵션이 불만족스러움
```

시나리오 3: 벤치마킹

```
-- 요구사항: "같은 업종 회사들과 '협력' 항목 비교"

-- 문제: 회사마다 '협력' 위치가 다름
-- A회사: category_1 = '협력'
-- B회사: category_1 = '조직문화', category_2 = '협력'
-- C회사: category_1 = '팀워크', category_2 = '협업' -- 다른 용어!

-- ✕ 표준화된 비교 불가능
-- 수동으로 매핑 테이블 관리 필요
```

2.4 유지보수 비용

정량적 비용 추정

```
신규 분류 추가 (1개):
- 스키마 검토: 0.5시간
```


- 영향도 분석: 1시간
- 코드 수정: 2시간
- 테스트: 1시간
- 배포: 0.5시간

총: 5시간/개

연간 신규 분류: 20-30개

연간 비용: 100-150시간 = 2-3주 개발 시간

5년 누적: 500-750시간 = 3-4개월 개발 리소스

정성적 비용

- 분류 체계 일관성 유지의 어려움
- 새 직원 온보딩시 학습 곡선
- 분류 변경시 기존 데이터 영향도 파악 어려움
- 애드혹 쿼리 작성 복잡도 증가

3. 제안 솔루션: 하이브리드 분류 체계

3.0 계층 구분의 명확화

중요한 깨달음

분류 체계를 설계할 때 두 가지 서로 다른 계층을 혼동하지 않아야 합니다:

계층 1: 명확한 메타데이터 (Structural Metadata)

특징:

- ☒ 명확하고 변하지 않음
- ☒ 상호 배타적 (하나만 선택)
- ☒ 시간이 지나도 의미 변화 없음
- ☒ 쿼리 필터링에 주로 사용

예시:

- 진단 유형: OD, LD, ES, MA
- 연도: 2023, 2024, 2025
- 회사: CJ, 삼성, LG
- 회사 규모: 대기업, 중견기업, 중소기업, 스타트업
- 업종: 제조, IT, 금융, 서비스
- 스케일: LIKERT_5, LIKERT_7

구현 방식:

- 일반 SQL 컬럼으로 구현
- 별도의 NOT NULL 컬럼
- CHECK 제약조건 사용
- B-Tree 인덱스

쿼리 예시:

```
SELECT * FROM questions
WHERE diagnosis_type = 'OD'
      AND year = 2024
      AND company_code = 'CJ';
```

계층 2: 의미론적 분류 (Semantic Taxonomy)

특징:

- ⚠️ 복잡하고 계속 진화함
- ⚠️ 다차원적 (여러 개 동시 가능)
- ⚠️ 시간에 따라 새로운 개념 등장
- ⚠️ 의미 기반 검색에 사용

예시:

- 주제(Themes): 조직문화, 리더십, 전략
- 개념(Concepts): 비전, 소통, 협력, 혁신
- 측면(Aspects): 이해도, 만족도, 중요도
- 대상(Subjects): 조직, 팀, 개인, 리더
- 트렌드(Trends): DX, ESG, AI, 메타버스

구현 방식:

- JSONB 태그 시스템
- 배열 형태로 여러 값 저장
- 동적으로 확장 가능
- GIN 인덱스

쿼리 예시:

```
SELECT * FROM questions
WHERE tags @> '{"concepts": ["비전"]}';
-- "비전"과 관련된 모든 개념도 자동 검색
```

하이브리드 접근법의 장점

```
-- 구조적 필터 + 의미론적 검색 조합
SELECT
  question_id,
  question_text,
  tags
FROM survey.question_tags
WHERE
  -- 명확한 메타데이터 필터 (빠른 필터링)
  diagnosis_type = 'OD'
  AND year = 2024
  AND company_code = 'CJ'

  -- 의미론적 태그 검색 (유연한 검색)
  AND (
```

```
tags @> '{"concepts": ["리더십"]}'  
OR tags @> '{"themes": ["조직문화"]}'  
);
```

-- 결과: 2024년 CJ 조직진단 중 리더십이나 조직문화 관련 문항

실제 사용 시나리오

시나리오 1: "2024년 CJ의 리더십 진단 점수"

- 명확한 메타데이터: year=2024, company_code='CJ', diagnosis_type='LD'
- 필요시 태그: tags @> '{"concepts": ["코칭", "피드백"]}'

시나리오 2: "디지털전환 관련 모든 문항"

- 의미론적 태그: tags @> '{"trends": ["DX"]}'
- 또는 tags @> '{"concepts": ["디지털전환"]}'
- 연도/회사 무관하게 모든 DX 관련 문항 검색

시나리오 3: "2023년과 2024년 비전 관련 점수 비교"

- 명확한 메타데이터: year IN (2023, 2024)
- 의미론적 태그: tags @> '{"concepts": ["비전"]}'
- 온톨로지가 자동으로 "미션", "방향성" 등 관련 개념도 포함

핵심 원칙

1. 명확하면 컬럼, 복잡하면 태그
2. 필터링은 컬럼, 검색은 태그
3. 정적이면 컬럼, 동적이면 태그
4. 배타적이면 컬럼, 다차원이면 태그

[REVISED] 특수 케이스: 척도(Scale) 정의

문제 재검토: "몰입도 척도 문항의 분포"

초기 접근 (오류):

- ✗ 별도 컬럼으로 명시:
measurement_purpose = 'engagement_scale'
→ 또 다른 고정 계층 추가 (확장성 문제)
- ✗ 의미론적 태그로만:
tags @> '{"concepts": ["몰입도"]}'
→ "몰입도 관련" 모든 문항 포함 (너무 광범위)

수정된 접근 (온톨로지 관계 기반):

- ☒ 온톨로지 관계로 척도 정의:
 - "몰입도" --HAS_COMPONENT--> "자극심"
 - "몰입도" --HAS_COMPONENT--> "소속감"
 - "몰입도" --HAS_COMPONENT--> "조직몰입"
- ☒ 쿼리시 온톨로지 관계를 따라감:
 1. 온톨로지에서 "몰입도"의 컴포넌트 추출
 2. 해당 컴포넌트를 concepts에 가진 문항 검색

핵심 깨달음

실무 현실:

- "이 문항은 몰입도 척도입니다" (X) - 명시적 라벨링 안 함
- "이 문항들이 의미론적으로 몰입도를 구성한다" (O) - 온톨로지 관계로 정의

계층 구분:

```
-- 예시 문항: "나는 우리 회사에서 일하는 것이 자랑스럽다"

-- =====
-- 계층 1: 구조적 메타데이터 (사실)
-- =====
diagnosis_type = 'ES' -- 몰입도 진단
year = 2024
company_code = 'CJ'
industry = '식품'

-- "이 문항은 2024년 CJ 몰입도 진단에서 사용되었다" (사실)

-- =====
-- 계층 2: 의미론적 분류 (개념)
-- =====
tags = {
  "concepts": ["자극심", "조직몰입", "정서적애탁"],
  "aspects": ["만족도", "소속감"],
  "subjects": ["개인", "조직"]
}

-- "이 문항은 자극심, 조직몰입 개념을 다룬다" (의미)

-- =====
-- 온톨로지 관계 (별도 테이블)
-- =====
taxonomy_relations:
  "몰입도" --HAS_COMPONENT--> "자극심"
  "몰입도" --HAS_COMPONENT--> "소속감"
  "몰입도" --HAS_COMPONENT--> "조직몰입"

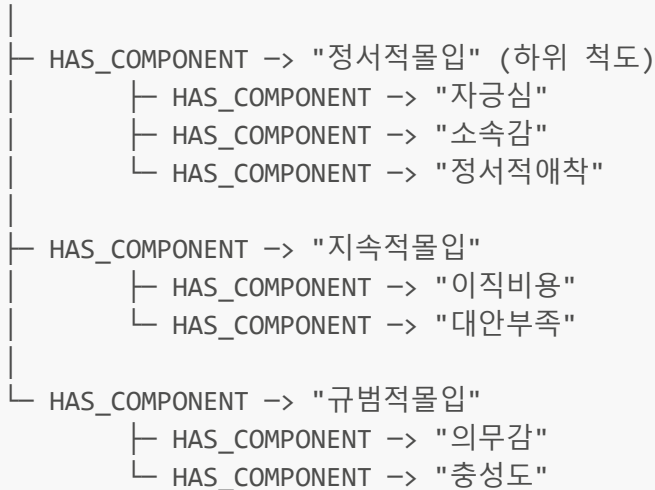
-- "몰입도는 자극심, 소속감 등으로 구성된다" (관계)
```

온톨로지 관계 기반 척도 정의

몰입도 척도의 온톨로지 구조:

온톨로지 그래프:

"몰입도" (상위 개념/척도)



실무 프로세스

1. 문항 생성 및 태깅:

```
-- 문항에는 의미론적 태그만 부여
INSERT INTO question_tags VALUES (
    'ES-2024-001',
    '우리 회사에서 일하는 것이 자랑스럽다',
    'ES', 2024, 'CJ', '식품',
    '{"concepts": ["자극심"], "aspects": ["만족도"]}':::jsonb
);
```

2. 온톨로지 관계 정의 (별도 작업):

```
-- taxonomy_relations 테이블에 관계 정의
INSERT INTO taxonomy_relations VALUES
    ('몰입도', '정서적몰입', 'HAS_COMPONENT', 1.0),
    ('정서적몰입', '자극심', 'HAS_COMPONENT', 1.0),
    ('정서적몰입', '소속감', 'HAS_COMPONENT', 1.0),
    ('정서적몰입', '정서적애착', 'HAS_COMPONENT', 1.0);
```

3. 척도 문항 조회:

```
-- "몰입도 척도 문항 분포" 쿼리
WITH RECURSIVE engagement_components AS (
  -- 기본: 몰입도의 직접 컴포넌트
  SELECT to_term as component, 1 as level
  FROM taxonomy_relations
  WHERE from_term = '몰입도'
  AND relation_type = 'HAS_COMPONENT'

  UNION ALL

  -- 재귀: 컴포넌트의 하위 컴포넌트
  SELECT tr.to_term, ec.level + 1
  FROM taxonomy_relations tr
  JOIN engagement_components ec ON tr.from_term = ec.component
  WHERE tr.relation_type = 'HAS_COMPONENT'
  AND ec.level < 3 -- 최대 깊이 제한
)
-- 해당 컴포넌트를 가진 문항들의 분포
SELECT
  jsonb_array_elements_text(tags->'concepts') as concept,
  COUNT(*) as question_count
FROM question_tags
WHERE tags->'concepts' ?| (
  SELECT array_agg(DISTINCT component) FROM engagement_components
)
GROUP BY concept
ORDER BY question_count DESC;

-- 결과:
-- concept          | question_count
-- -----+-----
-- 소속감           | 5
-- 자긍심           | 4
-- 정서적애착       | 4
-- 충성도           | 3
-- 의무감           | 2
```

핵심 장점

1. 유연성:

```
-- 2023년 몰입도 정의
'몰입도' → '자긍심', '소속감', '충성도'

-- 2024년 몰입도 정의 (심리적안전 추가)
INSERT INTO taxonomy_relations
VALUES ('정서적몰입', '심리적안전', 'HAS_COMPONENT', 1.0);

-- 문항 데이터 변경 없음! 관계만 추가
```

2. 재사용성:

```
-- "소속감"은 여러 척도에 사용 가능
'몰입도' → '정서적몰입' → '소속감'
'조직문화지수' → '응집력' → '소속감'

-- 같은 개념, 다른 맥락
```

3. 의미론적 일관성:

```
"몰입도"도 결국 하나의 concept
→ 별도 컬럼이 아니라 온톨로지의 일부
→ PARENT-CHILD 관계와 동일한 체계로 관리
```

시나리오별 쿼리

시나리오 1: "2024년 CJ의 몰입도 점수"

```
WITH engagement_components AS (
  -- 온톨로지에서 컴포넌트 추출 (재귀)
  ...
)
SELECT AVG(r.response_value) as engagement_score
FROM responses r
JOIN question_tags q ON r.question_id = q.question_id
WHERE q.year = 2024
      AND q.company_code = 'CJ'
      AND q.tags->'concepts' ?| (
        SELECT array_agg(component) FROM engagement_components
      );
```

시나리오 2: "정서적 몰입만 따로 계산"

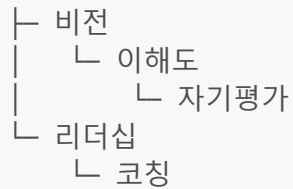
```
WITH affective_components AS (
  SELECT to_term as component
  FROM taxonomy_relations
  WHERE from_term = '정서적몰입'
        AND relation_type = 'HAS_COMPONENT'
)
SELECT AVG(r.response_value) as affective_score
FROM responses r
JOIN question_tags q ON r.question_id = q.question_id
WHERE q.tags->'concepts' ?| (
  SELECT array_agg(component) FROM affective_components
);
```

3.1 핵심 개념

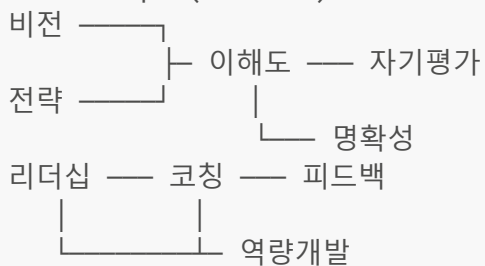
계층에서 네트워크로

기존: Tree (계층적)

조직진단



제안: Graph (네트워크)



장점:

- 다대다 관계 표현 가능
- 의미적 연결 명시적
- 계층 깊이 무관

태그 시스템

문항: "우리 조직의 디지털 전환 비전을 명확히 이해하고 있습니까?"

태그 (다차원):

```

{
  "themes": ["조직문화", "전략"],           // 주제
  "concepts": ["비전", "디지털전환"],       // 핵심 개념
  "aspects": ["이해도", "명확성"],          // 측정 측면
  "subjects": ["조직"],                     // 대상
  "methods": ["자기평가"],                  // 평가 방법
  "trends": ["DX", "2025"],                 // 트렌드/시대
  "custom": []                              // 고객사 특수
}
  
```

검색: "디지털전환" → 이 문항 포함

검색: "비전" → 이 문항 포함

검색: "조직문화" → 이 문항 포함

→ 다차원 검색 가능!

3.2 데이터베이스 스키마

핵심 테이블: question_tags

```

CREATE TABLE survey.question_tags (
  question_id VARCHAR(50) PRIMARY KEY,
  question_text TEXT NOT NULL,

  -- =====
  -- 1. 구조적 메타데이터 (사실 관계)
  -- =====
  diagnosis_type VARCHAR(20) NOT NULL, -- OD, LD, ES, MA (진단 유형)
  year INT, -- 진단 연도 (2023, 2024, 2025)
  company_code VARCHAR(10), -- 회사 코드 (CJ, SAMS, etc)
  company_size VARCHAR(20), -- 회사 규모 (대기업, 중견기업, 중소기업,
스타트업)
  industry VARCHAR(50), -- 업종 (제조, IT, 금융)
  scale_type VARCHAR(20), -- LIKERT_5, LIKERT_7 등

  -- =====
  -- 2. 의미론적 태그 (개념/의미)
  -- =====
  tags JSONB NOT NULL,
  /*
  구조:
  {
    "themes": ["조직문화", "리더십"], -- 대주제 (의미론적)
    "concepts": ["비전", "소통", "협력"], -- 핵심 개념 (의미론적)
    "aspects": ["이해도", "만족도"], -- 측정 측면 (의미론적)
    "subjects": ["조직", "팀", "리더"], -- 평가 대상 (의미론적)
    "methods": ["자기평가", "타인평가"], -- 평가 방법 (의미론적)
    "trends": ["DX", "ESG", "AI"], -- 시대적 트렌드
    "custom": { -- 회사별 특수 태그
      "cj_core_value": "ONLYONE",
      "custom_category": "..."}
  }
  */

  NOTE:
  - 명확한 분류(연도, 회사, 업종)는 별도 컬럼
  - 복잡하고 진화하는 의미론적 분류는 JSONB 태그
  - 척도 정의는 taxonomy_relations 테이블의 온톨로지 관계로 처리
  */

  -- =====
  -- 3. 검색 최적화
  -- =====
  tag_search_vector tsvector GENERATED ALWAYS AS (
    to_tsvector('korean', tags::text || ' ' || question_text)
  ) STORED,

  -- =====
  -- 4. 자동 분류 메타데이터
  -- =====

```

```

auto_tagged BOOLEAN DEFAULT FALSE,
tag_confidence FLOAT, -- 0-1, 자동 태깅 신뢰도

created_at TIMESTAMP DEFAULT NOW(),
updated_at TIMESTAMP DEFAULT NOW(),

-- =====
-- 5. 제약조건
-- =====
CONSTRAINT valid_diagnosis_type CHECK (
    diagnosis_type IN ('OD', 'LD', 'ES', 'MA')
),
CONSTRAINT valid_year CHECK (
    year IS NULL OR (year >= 2000 AND year <= 2100)
)
);

-- =====
-- 인덱스
-- =====
-- 1. 구조적 메타데이터 인덱스 (빠른 필터링)
CREATE INDEX idx_diagnosis_type ON survey.question_tags(diagnosis_type);
CREATE INDEX idx_year ON survey.question_tags(year);
CREATE INDEX idx_company ON survey.question_tags(company_code);
CREATE INDEX idx_company_size ON survey.question_tags(company_size);
CREATE INDEX idx_industry ON survey.question_tags(industry);

-- 복합 인덱스 (자주 사용하는 조합)
CREATE INDEX idx_diagnosis_year ON survey.question_tags(diagnosis_type, year);
CREATE INDEX idx_company_year ON survey.question_tags(company_code, year);
CREATE INDEX idx_size_industry ON survey.question_tags(company_size, industry);

-- 2. JSONB 태그 인덱스 (의미론적 검색)
CREATE INDEX idx_tags_gin ON survey.question_tags USING GIN(tags);

-- 3. 전문 검색 인덱스
CREATE INDEX idx_tag_search ON survey.question_tags USING GIN(tag_search_vector);

```

온톨로지 테이블: taxonomy

```

CREATE TABLE survey.taxonomy (
    taxonomy_id SERIAL PRIMARY KEY,

    -- 분류 용어
    term VARCHAR(100) UNIQUE NOT NULL,
    term_type VARCHAR(20) NOT NULL, -- THEME, CONCEPT, ASPECT, SUBJECT, METHOD

    -- 다국어 및 동의어
    aliases JSONB,
    /*
    {

```

```

"ko": ["비전", "vision", "미래상", "미래비전"],
"en": ["vision", "future vision"],
"official": "비전", // 공식 표준 용어
"deprecated": ["미래상"] // 더 이상 사용 안함
}
*/

-- 의미론적 관계 (Graph Edges)
parent_terms JSONB,      -- 상위 개념들 ["전략", "조직문화"]
child_terms JSONB,       -- 하위 개념들 ["미션", "목표", "방향성"]
related_terms JSONB,     -- 연관 개념들 ["전략", "목표", "방향"]
similar_terms JSONB,     -- 유사 개념들 (동의어 그룹)

-- 설명
description TEXT,
examples TEXT[], -- 예시 문항들

-- 통계
usage_count INT DEFAULT 0,      -- 사용 빈도
first_used_year INT,            -- 처음 등장 연도
trend_score FLOAT,              -- 트렌드 점수 (최근 증가율)

-- 가중치
importance_weight FLOAT DEFAULT 1.0, -- 중요도

-- 메타
created_at TIMESTAMP DEFAULT NOW(),
updated_at TIMESTAMP DEFAULT NOW(),
created_by VARCHAR(50),

CONSTRAINT valid_term_type CHECK (
    term_type IN ('THEME', 'CONCEPT', 'ASPECT', 'SUBJECT', 'METHOD',
'INDUSTRY', 'TREND', 'CUSTOM')
)
);

-- 인덱스
CREATE INDEX idx_term_type ON survey.taxonomy(term_type);
CREATE INDEX idx_aliases_gin ON survey.taxonomy USING GIN(aliases);
CREATE INDEX idx_usage_count ON survey.taxonomy(usage_count DESC);
CREATE INDEX idx_trend ON survey.taxonomy(trend_score DESC NULLS LAST);

```

관계 테이블: taxonomy_relations

```

CREATE TABLE survey.taxonomy_relations (
    relation_id SERIAL PRIMARY KEY,

    from_term VARCHAR(100) REFERENCES survey.taxonomy(term),
    to_term VARCHAR(100) REFERENCES survey.taxonomy(term),

    relation_type VARCHAR(20) NOT NULL,

```

```

-- PARENT, CHILD, RELATED, SIMILAR, SYNONYM, OPPOSITE, PART_OF, INSTANCE_OF

strength FLOAT DEFAULT 1.0, -- 관계 강도 (0-1)

-- 관계 메타데이터
context JSONB, -- 어떤 맥락에서 이 관계가 성립하는지
/*
{
  "domain": "leadership", // 리더십 도메인에서만
  "year_from": 2020,      // 2020년부터
  "companies": ["CJ"]     // CJ에서만
}
*/

created_at TIMESTAMP DEFAULT NOW(),
created_by VARCHAR(50),

UNIQUE(from_term, to_term, relation_type)
);

CREATE INDEX idx_from_term ON survey.taxonomy_relations(from_term);
CREATE INDEX idx_to_term ON survey.taxonomy_relations(to_term);
CREATE INDEX idx_relation_type ON survey.taxonomy_relations(relation_type);

```

진화 추적 테이블

```

CREATE TABLE survey.taxonomy_evolution (
  evolution_id SERIAL PRIMARY KEY,

  term VARCHAR(100),
  change_type VARCHAR(20) NOT NULL,
  -- ADDED, REMOVED, RENAMED, MERGED, SPLIT, DEPRECATED, REVIVED

  old_value JSONB,
  new_value JSONB,

  reason TEXT,
  affected_questions INT, -- 영향받은 문항 수

  changed_at TIMESTAMP DEFAULT NOW(),
  changed_by VARCHAR(50)
);

```

3.3 동적 계층 생성

Python 구현

```

import networkx as nx
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

class DynamicTaxonomyManager:
    """분류 체계를 동적으로 생성하고 관리"""

    def __init__(self, db_connection):
        self.db = db_connection
        self.taxonomy_graph = None
        self.embedding_model = None
        self._load_taxonomy()

    def _load_taxonomy(self):
        """데이터베이스에서 온톨로지 로드"""
        # 1. 모든 분류 용어 로드
        terms = self.db.execute("""
            SELECT term, term_type, aliases, parent_terms,
                   related_terms, importance_weight
            FROM survey.taxonomy
        """).fetchall()

        # 2. NetworkX 그래프 생성
        self.taxonomy_graph = nx.DiGraph()

        for term_data in terms:
            self.taxonomy_graph.add_node(
                term_data['term'],
                **term_data
            )

            # 관계 추가
            if term_data['parent_terms']:
                for parent in term_data['parent_terms']:
                    self.taxonomy_graph.add_edge(
                        parent,
                        term_data['term'],
                        relation='parent',
                        weight=1.0
                    )

            if term_data['related_terms']:
                for related in term_data['related_terms']:
                    self.taxonomy_graph.add_edge(
                        term_data['term'],
                        related,
                        relation='related',
                        weight=0.7
                    )

        print(f"☑ 온톨로지 로드 완료: {len(terms)} 용어, "
              f"{self.taxonomy_graph.number_of_edges()} 관계")

```

```

def auto_tag_question(self, question_text, max_tags=10):
    """
    문항 텍스트에서 자동으로 태그 추출

    Args:
        question_text: 문항 텍스트
        max_tags: 최대 태그 수

    Returns:
        dict: 다차원 태그
    """
    detected_tags = {
        'themes': [],
        'concepts': [],
        'aspects': [],
        'subjects': [],
        'methods': [],
        'trends': [],
        'companies': [],
        'custom': {}
    }

    # 1. 명시적 키워드 매칭
    for term_node in self.taxonomy_graph.nodes():
        term_data = self.taxonomy_graph.nodes[term_node]

        # 본명 + 모든 별칭
        all_names = [term_node]
        if term_data.get('aliases'):
            all_names.extend(term_data['aliases'].get('ko', []))

        # 텍스트에 포함되어 있는지
        for name in all_names:
            if name in question_text:
                term_type = term_data['term_type']
                category = self._type_to_category(term_type)

                if term_node not in detected_tags[category]:
                    detected_tags[category].append(term_node)
                break

    # 2. 의미론적 유사도 기반 태깅 (임베딩)
    semantic_tags = self._semantic_tagging(question_text, top_k=5)

    # 병합
    for tag_type, tags in semantic_tags.items():
        for tag in tags:
            if tag not in detected_tags[tag_type]:
                detected_tags[tag_type].append(tag)

    # 3. 관련 용어 확장 (그래프 기반)
    detected_tags = self._expand_tags(detected_tags, depth=1)

```

```

# 4. 신뢰도 계산
confidence = self._calculate_confidence(detected_tags, question_text)

return {
    'tags': detected_tags,
    'confidence': confidence,
    'auto_generated': True
}

def _semantic_tagging(self, question_text, top_k=5):
    """의미론적 유사도 기반 태깅"""
    if self.embedding_model is None:
        self.embedding_model = SentenceTransformer(
            'sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2'
        )

    # 문항 임베딩
    question_emb = self.embedding_model.encode([question_text])

    # 모든 분류 용어 임베딩
    all_terms = list(self.taxonomy_graph.nodes())
    term_embeddings = self.embedding_model.encode(all_terms)

    # 유사도 계산
    similarities = cosine_similarity(question_emb, term_embeddings)[0]

    # 상위 k개
    top_indices = similarities.argsort()[-top_k:][::-1]

    result = {
        'concepts': [],
        'themes': [],
        'aspects': []
    }

    for idx in top_indices:
        if similarities[idx] > 0.5: # 임계값
            term = all_terms[idx]
            term_type = self.taxonomy_graph.nodes[term]['term_type']
            category = self._type_to_category(term_type)
            result[category].append(term)

    return result

def _expand_tags(self, tags, depth=1):
    """그래프 기반 태그 확장"""
    expanded = tags.copy()

    for category, tag_list in tags.items():
        for tag in tag_list:
            if tag in self.taxonomy_graph:
                # 직계 부모/자식 추가
                parents = list(self.taxonomy_graph.predecessors(tag))
                children = list(self.taxonomy_graph.successors(tag))

```

```

        # themes에 부모 추가
        if category == 'concepts' and parents:
            expanded['themes'].extend(
                p for p in parents
                if p not in expanded['themes']
            )

    return expanded

def _calculate_confidence(self, tags, question_text):
    """태깅 신뢰도 계산"""
    total_tags = sum(len(v) if isinstance(v, list) else 0
                     for v in tags.values())

    if total_tags == 0:
        return 0.0

    # 명시적 매칭 비율
    explicit_matches = sum(
        1 for cat_tags in tags.values()
        if isinstance(cat_tags, list)
        for tag in cat_tags
        if tag in question_text
    )

    explicit_ratio = explicit_matches / total_tags

    # 간단한 휴리스틱
    if explicit_ratio > 0.7:
        return 0.9
    elif explicit_ratio > 0.4:
        return 0.7
    else:
        return 0.5

def _type_to_category(self, term_type):
    """term_type → 태그 카테고리 매핑"""
    mapping = {
        'THEME': 'themes',
        'CONCEPT': 'concepts',
        'ASPECT': 'aspects',
        'SUBJECT': 'subjects',
        'METHOD': 'methods',
        'TREND': 'trends',
        'CUSTOM': 'custom'
    }
    return mapping.get(term_type, 'custom')

def suggest_new_term(self, term, sample_questions=None):
    """신규 분류 용어 등록 제안"""
    # 1. 유사 용어 검색
    similar_terms = self._find_similar_terms(term, top_k=5)

```



```

# 2. 용어 타입 추론
inferred_type = self._infer_term_type(term, sample_questions)

# 3. 배치 위치 추론
suggested_parents = []
suggested_related = []

for similar in similar_terms:
    # 유사 용어의 이웃들
    neighbors = list(self.taxonomy_graph.neighbors(similar['term']))
    suggested_parents.extend(neighbors)

# 빈도순 정렬
from collections import Counter
parent_counts = Counter(suggested_parents)
top_parents = [term for term, _ in parent_counts.most_common(3)]

return {
    'new_term': term,
    'suggested_type': inferred_type,
    'similar_terms': similar_terms,
    'suggested_parents': top_parents,
    'suggested_related': list(set(suggested_related)),
    'confidence': np.mean([s['similarity'] for s in similar_terms]) if
similar_terms else 0.0
}

def _find_similar_terms(self, term, top_k=5):
    """의미론적 유사 용어 검색"""
    if self.embedding_model is None:
        self.embedding_model = SentenceTransformer(
            'sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2'
        )

    # 신규 용어 임베딩
    new_emb = self.embedding_model.encode([term])

    # 기존 용어들
    existing_terms = list(self.taxonomy_graph.nodes())
    existing_embs = self.embedding_model.encode(existing_terms)

    # 유사도
    similarities = cosine_similarity(new_emb, existing_embs)[0]

    # 상위 k개
    top_indices = similarities.argsort()[-top_k:][::-1]

    return [
        {
            'term': existing_terms[i],
            'similarity': float(similarities[i])
        }
        for i in top_indices
        if similarities[i] > 0.3 # 최소 임계값
    ]

```

```

]

def _infer_term_type(self, term, sample_questions):
    """용어 타입 추론 (THEME, CONCEPT, ASPECT 등)"""
    # 간단한 휴리스틱

    # 측정 관련 단어 → ASPECT
    aspect_keywords = ['도', '성', '률', '수준', '정도', '만족', '이해', '중요']
    if any(kw in term for kw in aspect_keywords):
        return 'ASPECT'

    # 대상 관련 → SUBJECT
    subject_keywords = ['조직', '팀', '개인', '리더', '직원', '임원']
    if any(kw in term for kw in subject_keywords):
        return 'SUBJECT'

    # 방법 관련 → METHOD
    method_keywords = ['평가', '측정', '진단', '분석']
    if any(kw in term for kw in method_keywords):
        return 'METHOD'

    # 기본: CONCEPT
    return 'CONCEPT'

def build_dynamic_hierarchy(self, question_id, max_depth=4):
    """
    특정 문항에 대해 동적으로 계층 생성

    Returns:
        list: 계층 경로 (예: ['조직문화', '협력', '팀워크', '심리적안전'])
    """
    # 1. 문항의 태그 가져오기
    question = self.db.execute("""
        SELECT tags FROM survey.question_tags WHERE question_id = %s
    """, (question_id,)).fetchone()

    if not question:
        return []

    tags = question['tags']

    # 2. 계층 구성 알고리즘
    hierarchy = []

    # 우선순위: themes > concepts > aspects > subjects
    for category in ['themes', 'concepts', 'aspects', 'subjects']:
        if category in tags and tags[category]:
            # 가장 중요한 것 선택 (가중치 기반)
            best_term = self._select_most_important(tags[category])
            if best_term:
                hierarchy.append(best_term)

        if len(hierarchy) >= max_depth:
            break

```

```

        return hierarchy

def _select_most_important(self, terms):
    """용어 리스트에서 가장 중요한 것 선택"""
    if not terms:
        return None

    # 가중치 기반
    term_weights = []
    for term in terms:
        if term in self.taxonomy_graph:
            weight = self.taxonomy_graph.nodes[term].get('importance_weight',
1.0)

            term_weights.append((term, weight))

    if not term_weights:
        return terms[0]

    # 가중치 최대
    return max(term_weights, key=lambda x: x[1])[0]

```

3.4 검색 및 쿼리

고급 검색 기능

```

class TaxonomySearchEngine:
    """온톨로지 기반 지능형 검색"""

    def __init__(self, db, taxonomy_manager):
        self.db = db
        self.taxonomy = taxonomy_manager

    def search_questions(self, query, filters=None, expand=True):
        """
        자연어 쿼리로 문항 검색

        Args:
            query: 검색어 (예: "리더십 관련 문항")
            filters: 추가 필터 (diagnosis_type, year 등)
            expand: 관련 용어로 확장 검색할지

        Returns:
            list: 검색 결과
        """
        # 1. 쿼리에서 분류 용어 추출
        query_tags = self.taxonomy.auto_tag_question(query)

        # 2. 관련 용어 확장
        if expand:
            expanded_terms = self._expand_query_terms(query_tags['tags'])

```

```

else:
    expanded_terms = query_tags['tags']

# 3. PostgreSQL JSONB 쿼리
conditions = []
params = []

for category, terms in expanded_terms.items():
    if terms and isinstance(terms, list):
        # JSONB 배열에 포함되는지 확인
        conditions.append(f"tags->'{{category}}' ?| %s")
        params.append(terms)

where_clause = " OR ".join(conditions) if conditions else "TRUE"

# 4. 검색 실행
query_sql = f"""
    SELECT
        question_id,
        question_text,
        tags,
        ts_rank(tag_search_vector, plainto_tsquery('korean', %s)) as
relevance
    FROM survey.question_tags
    WHERE ({where_clause})
    """

params.insert(0, query)

if filters:
    if 'diagnosis_type' in filters:
        query_sql += " AND diagnosis_type = %s"
        params.append(filters['diagnosis_type'])

query_sql += " ORDER BY relevance DESC LIMIT 100"

results = self.db.execute(query_sql, params).fetchall()

return results

def _expand_query_terms(self, tags):
    """쿼리 용어를 관련 용어로 확장"""
    expanded = {}

    for category, terms in tags.items():
        if not isinstance(terms, list):
            continue

        expanded[category] = list(terms) # 복사

        for term in terms:
            # 동의어 추가
            synonyms = self._get_synonyms(term)
            expanded[category].extend(synonyms)

```

```

        # 직계 자식 추가 (하위 개념)
        if category == 'concepts':
            children = self._get_children(term)
            expanded[category].extend(children)

# 중복 제거
for category in expanded:
    expanded[category] = list(set(expanded[category]))

return expanded

def _get_synonyms(self, term):
    """용어의 동의어 가져오기"""
    result = self.db.execute("""
        SELECT aliases FROM survey.taxonomy WHERE term = %s
    """, (term,)).fetchone()

    if result and result['aliases']:
        return result['aliases'].get('ko', [])
    return []

def _get_children(self, term):
    """하위 개념 가져오기"""
    if term in self.taxonomy.taxonomy_graph:
        return list(self.taxonomy.taxonomy_graph.successors(term))
    return []

def find_similar_questions(self, question_id, top_k=10, threshold=0.5):
    """유사 문항 찾기"""
    # 1. 대상 문항 태그
    target = self.db.execute("""
        SELECT tags FROM survey.question_tags WHERE question_id = %s
    """, (question_id,)).fetchone()

    if not target:
        return []

    target_tags = target['tags']

    # 2. 태그 유사도 기반 검색
    # Jaccard 유사도 활용
    results = self.db.execute("""
        WITH target_tags AS (
            SELECT
                jsonb_array_elements_text(tags->'concepts') ||
                jsonb_array_elements_text(tags->'aspects') as tag
            FROM survey.question_tags
            WHERE question_id = %s
        ),
        candidate_tags AS (
            SELECT
                question_id,
                jsonb_array_elements_text(tags->'concepts') ||
    """

```

```

        jsonb_array_elements_text(tags->'aspects') as tag
    FROM survey.question_tags
    WHERE question_id != %s
)
SELECT
    c.question_id,
    COUNT(DISTINCT CASE WHEN c.tag IN (SELECT tag FROM target_tags)
THEN c.tag END)::float /
    COUNT(DISTINCT c.tag)::float as similarity
FROM candidate_tags c
GROUP BY c.question_id
HAVING COUNT(DISTINCT CASE WHEN c.tag IN (SELECT tag FROM target_tags)
THEN c.tag END) > 0
ORDER BY similarity DESC
LIMIT %s
"""', (question_id, question_id, top_k)).fetchall()

return [r for r in results if r['similarity'] >= threshold]

```

3.5 관리 및 진화

온톨로지 관리 도구

```

class TaxonomyManager:
    """온톨로지 관리 및 진화"""

    def add_new_term(self, term, term_type, metadata=None):
        """신규 용어 추가"""
        # 1. 중복 체크
        existing = self.db.execute("""
            SELECT term FROM survey.taxonomy WHERE term = %s
        """, (term,)).fetchone()

        if existing:
            return {'status': 'error', 'message': '이미 존재하는 용어'}

        # 2. 삽입
        self.db.execute("""
            INSERT INTO survey.taxonomy (
                term, term_type, aliases, description, first_used_year
            ) VALUES (%s, %s, %s, %s, %s)
        """, (
            term,
            term_type,
            metadata.get('aliases', {}),
            metadata.get('description', ''),
            metadata.get('year', 2025)
        ))

        # 3. 진화 기록
        self.db.execute("""

```

```

        INSERT INTO survey.taxonomy_evolution (
            term, change_type, new_value, reason
        ) VALUES (%s, 'ADDED', %s, %s)
    """ % (term, {'term_type': term_type}, metadata.get('reason', '')))

# 4. 그래프 갱신
self.reload_taxonomy()

return {'status': 'success', 'term': term}

def merge_terms(self, term1, term2, keep=None):
    """
    중복 용어 병합

    Args:
        term1, term2: 병합할 용어
        keep: 유지할 용어 (None이면 term1)
    """
    keep_term = keep or term1
    remove_term = term2 if keep_term == term1 else term1

    # 1. remove_term을 keep_term의 별칭으로
    self.db.execute("""
        UPDATE survey.taxonomy
        SET aliases = jsonb_set(
            COALESCE(aliases, '{} '::jsonb),
            '{ko}',
            COALESCE(aliases->'ko', '[] '::jsonb) || %s::jsonb
        )
        WHERE term = %s
    """, ([remove_term], keep_term))

    # 2. remove_term을 사용하는 모든 문항 업데이트
    self.db.execute("""
        UPDATE survey.question_tags
        SET tags = (
            SELECT jsonb_object_agg(
                key,
                CASE
                    WHEN jsonb_typeof(value) = 'array' THEN
                        (SELECT jsonb_agg(
                            CASE WHEN elem = %s THEN %s ELSE elem END
                        ) FROM jsonb_array_elements_text(value) elem)
                    ELSE value
                END
            )
            FROM jsonb_each(tags)
        )
        WHERE tags::text LIKE %s
    """, (remove_term, keep_term, f'%{remove_term}%'))

    # 3. 진화 기록
    self.db.execute("""
        INSERT INTO survey.taxonomy_evolution (

```

```

        term, change_type, old_value, new_value, reason
    ) VALUES (%s, 'MERGED', %s, %s, 'Term consolidation')
    """ , (keep_term, {'merged_from': remove_term}, {'into': keep_term}))

# 4. remove_term 삭제
self.db.execute("""
    DELETE FROM survey.taxonomy WHERE term = %s
    """ , (remove_term,))

return {'status': 'success', 'kept': keep_term, 'removed': remove_term}

def suggest_merges(self, threshold=0.85):
    """병합 가능한 중복 용어 제안"""
    # 모든 용어 쌍의 유사도 계산
    terms = self.db.execute("""
        SELECT term FROM survey.taxonomy
    """).fetchall()

    suggestions = []

    from sentence_transformers import SentenceTransformer
    from sklearn.metrics.pairwise import cosine_similarity

    model = SentenceTransformer('paraphrase-multilingual-MiniLM-L12-v2')
    term_list = [t['term'] for t in terms]
    embeddings = model.encode(term_list)

    similarities = cosine_similarity(embeddings)

    for i in range(len(term_list)):
        for j in range(i+1, len(term_list)):
            if similarities[i][j] > threshold:
                suggestions.append({
                    'term1': term_list[i],
                    'term2': term_list[j],
                    'similarity': float(similarities[i][j]),
                    'suggested_action': 'merge'
                })

    return sorted(suggestions, key=lambda x: x['similarity'], reverse=True)

def analyze_usage_trends(self, years=5):
    """분류 용어 사용 추세 분석"""
    result = self.db.execute("""
        WITH yearly_usage AS (
            SELECT
                jsonb_array_elements_text(tags->'concepts') as term,
                EXTRACT(YEAR FROM created_at) as year,
                COUNT(*) as usage_count
            FROM survey.question_tags
            WHERE created_at >= NOW() - INTERVAL '%s years'
            GROUP BY term, year
        )
        SELECT
    """

```



```

        term,
        jsonb_object_agg(year, usage_count) as yearly_counts,
        SUM(usage_count) as total_usage,
        -- 증가율 계산 (최근년도 / 과거년도)
        MAX(CASE WHEN year = EXTRACT(YEAR FROM NOW()) THEN usage_count
END)::float /
        NULLIF(MIN(CASE WHEN year = EXTRACT(YEAR FROM NOW()) - %s THEN
usage_count END)::float, 0) as growth_rate
        FROM yearly_usage
        GROUP BY term
        ORDER BY growth_rate DESC NULLS LAST
        """, (years, years-1)).fetchall()

    return result

```

4. 기술 구현 상세

4.1 자동 태깅 파이프라인

전체 흐름

```

[신규 문항 입력]
  ↓
[텍스트 전처리]
  - 정규화
  - 형태소 분석
  ↓
[1차: 명시적 매칭]
  - 온톨로지 용어 직접 검색
  - 별칭/동의어 매칭
  ↓
[2차: 의미론적 매칭]
  - 임베딩 기반 유사도
  - 상위 k개 후보
  ↓
[3차: 그래프 확장]
  - 부모/자식 관계 활용
  - 관련 개념 추가
  ↓
[신뢰도 계산]
  - 명시적 vs 추론 비율
  - 일관성 검증
  ↓
[수동 검토 여부 결정]
  confidence > 0.8: 자동 적용
  confidence ≤ 0.8: 수동 검토 필요
  ↓
[DB 저장]

```

구현 코드

```

class AutoTaggingPipeline:
    """자동 태깅 파이프라인"""

    def __init__(self, taxonomy_manager):
        self.taxonomy = taxonomy_manager
        self.nlp = self._init_nlp()

    def _init_nlp(self):
        """한국어 NLP 초기화"""
        from konlpy.tag import Okt
        return Okt()

    def process_question(self, question_text, manual_review_threshold=0.8):
        """문항 처리"""
        # 1. 전처리
        normalized = self._normalize(question_text)
        nouns = self.nlp.nouns(normalized)

        # 2. 자동 태깅
        tags_result = self.taxonomy.auto_tag_question(question_text)
        tags = tags_result['tags']
        confidence = tags_result['confidence']

        # 3. 검증
        validation = self._validate_tags(tags, question_text)

        # 4. 결정
        if confidence >= manual_review_threshold and validation['passed']:
            status = 'AUTO_APPROVED'
        else:
            status = 'MANUAL_REVIEW_REQUIRED'

        return {
            'tags': tags,
            'confidence': confidence,
            'validation': validation,
            'status': status,
            'review_reason': validation.get('issues', [])
        }

    def _normalize(self, text):
        """텍스트 정규화"""
        import re
        # 공백 정규화
        text = re.sub(r'\s+', ' ', text)
        # 특수문자 제거 (필요한 것만 유지)
        text = re.sub(r'^\w\s가-힣?', '', text)
        return text.strip()

    def _validate_tags(self, tags, question_text):

```

```

"""태그 검증"""
issues = []

# 1. 최소 태그 수
total_tags = sum(len(v) if isinstance(v, list) else 0 for v in
tags.values())
if total_tags < 2:
    issues.append('태그 수 부족 (최소 2개 필요)')

# 2. 필수 카테고리
if not tags.get('concepts'):
    issues.append('핵심 개념(concepts) 누락')

# 3. 중복 검사
all_tags_flat = []
for v in tags.values():
    if isinstance(v, list):
        all_tags_flat.extend(v)

if len(all_tags_flat) != len(set(all_tags_flat)):
    issues.append('중복 태그 존재')

return {
    'passed': len(issues) == 0,
    'issues': issues
}

```

4.2 성능 최적화

PostgreSQL 설정

```

-- 1. GIN 인덱스 최적화
CREATE INDEX idx_tags_gin ON survey.question_tags
USING GIN(tags jsonb_path_ops); -- path_ops for faster containment

-- 2. 부분 인덱스 (자주 검색하는 진단 유형만)
CREATE INDEX idx_tags_od ON survey.question_tags
USING GIN(tags)
WHERE diagnosis_type = 'OD';

-- 3. 텍스트 검색 인덱스
CREATE INDEX idx_text_search ON survey.question_tags
USING GIN(tag_search_vector);

-- 4. 복합 인덱스
CREATE INDEX idx_type_tags ON survey.question_tags(diagnosis_type, tags);

```

캐싱 전략

```

from functools import lru_cache
import redis

class CachedTaxonomyManager(DynamicTaxonomyManager):
    """캐싱이 적용된 온톨로지 매니저"""

    def __init__(self, db_connection, redis_client=None):
        super().__init__(db_connection)
        self.redis = redis_client or redis.Redis()
        self.cache_ttl = 3600 # 1시간

    @lru_cache(maxsize=1000)
    def get_term_info(self, term):
        """용어 정보 (메모리 캐시)"""
        if term in self.taxonomy_graph:
            return self.taxonomy_graph.nodes[term]
        return None

    def auto_tag_question(self, question_text, max_tags=10):
        """자동 태깅 (Redis 캐시)"""
        # 캐시 키
        cache_key = f"autotag:{hash(question_text)}"

        # 캐시 확인
        cached = self.redis.get(cache_key)
        if cached:
            import json
            return json.loads(cached)

        # 계산
        result = super().auto_tag_question(question_text, max_tags)

        # 캐시 저장
        self.redis.setex(
            cache_key,
            self.cache_ttl,
            json.dumps(result, ensure_ascii=False)
        )

        return result

```

4.3 확장성 고려

샤딩 전략 (미래)

```

-- 진단 유형별 파티셔닝 (100만+ 문항 대비)
CREATE TABLE survey.question_tags_od PARTITION OF survey.question_tags
FOR VALUES IN ('OD');

CREATE TABLE survey.question_tags_ld PARTITION OF survey.question_tags

```

```
FOR VALUES IN ('LD');

CREATE TABLE survey.question_tags_es PARTITION OF survey.question_tags
FOR VALUES IN ('ES');
```

5. 비교 분석 및 장단점

5.1 상세 비교표

측면	고정 계층 방식	태그 기반 온톨로지
확장성	⚠ 스키마 변경 필요	☒ 데이터만 추가
유연성	✗ 고정된 깊이만 가능	☒ 2-10단계 자유
다차원 분류	✗ 불가능	☒ 무제한 태그
의미 검색	✗ 정확 일치만	☒ 관련 개념 자동
동의어 처리	✗ 수동 관리	☒ 온톨로지 내장
신규 개념	⚠ 회의 및 승인	☒ 즉시 추가
학습 곡선	☒ 낮음	⚠ 중간
쿼리 복잡도	☒ 단순	⚠ JSONB 필요
초기 구축	☒ 빠름	⚠ 온톨로지 필요
장기 유지보수	✗ 증가	☒ 감소
데이터 품질	⚠ 일관성 유지 어려움	☒ 자동 검증
벤치마킹	✗ 표준화 어려움	☒ 자동 매핑

5.2 장단점 분석

태그 기반 온톨로지의 장점

1. 무한 확장성

```
# 10년 후 "메타버스 조직문화" 개념 등장
add_term("메타버스조직문화", "CONCEPT")
# → 기존 코드 변경 없음!
```

2. 다차원 분류

```
{
  "concepts": ["리더십", "디지털전환"],
  "aspects": ["역량", "실행력"],
```

```
"subjects": ["팀", "조직"]
}
```

// 한 문항이 6개 차원에 동시 속함

3. 자동 진화

```
# 사용 빈도, 트렌드 자동 추적
# 관련 개념 자동 제안
# 중복 용어 자동 감지
```

4. 의미 기반 검색

```
search("리더십")
# → "코칭", "피드백", "권한위임"도 자동 포함
```

태그 기반 온톨로지의 단점

1. 초기 구축 비용

- 온톨로지 설계 필요 (50-100개 핵심 용어)
- 과거 문항 재태깅 (수백~수천 개)
- 예상 시간: 1-2주

2. 학습 곡선

- 팀원들이 새로운 개념 이해 필요
- JSONB 쿼리 작성법 학습
- 예상 시간: 1-2주 온보딩

3. 데이터 품질 의존성

- 태그 품질이 중요
- 자동 태깅 신뢰도 관리 필요
- 정기적 검토 필요

4. 복잡도

- 단순 SELECT보다 복잡한 쿼리
- 그래프 관계 이해 필요

5.3 ROI 분석

비용

```
초기 구축 (1회):
- 온톨로지 설계: 2-3일
```

- 시스템 개발: 2-3주
 - 과거 데이터 마이그레이션: 1주
 - 테스트 및 검증: 1주
- 총: 5-6주

연간 운영:

- 온톨로지 관리: 월 2시간 = 24시간/년
 - 검토 및 보정: 분기 1일 = 4일/년
- 총: 약 1주/년

편익 (고정 계층 대비)

연간 절감:

- 신규 분류 추가: 20개 × 5시간 = 100시간
 - 분류 변경 작업: 10회 × 3시간 = 30시간
 - 검색 쿼리 작성: 50회 × 0.5시간 = 25시간
 - 벤치마킹 매핑: 20개 × 2시간 = 40시간
- 총: 195시간/년 = 약 5주

ROI: $(195 - 32) / 240 = 68\%$ 시간 절감

Break-even

- 초기 투자: 6주
- 연간 절감: 5주
- **Break-even: 약 1.2년**
- 이후 매년 5주씩 순 절감

6. 마이그레이션 전략

6.1 단계별 전환 계획

Phase 0: 준비 (1주)

- [] 기존 분류 체계 문서화
- [] 과거 문항 500-1000개 샘플링
- [] 핵심 분류 용어 50-100개 추출
- [] 온톨로지 초안 설계

Phase 1: 온톨로지 구축 (1-2주)

```
# 1. 빈도 분석
past_questions = load_historical_questions(limit=1000)
term_frequency = analyze_term_frequency(past_questions)
```

```
# 2. 핵심 용어 추출
core_terms = term_frequency.most_common(100)

# 3. 수동 분류
for term, count in core_terms:
    term_type = manual_classify(term) # THEME, CONCEPT, ASPECT
    add_to_taxonomy(term, term_type, usage_count=count)

# 4. 관계 설정
for term in core_terms:
    parents = identify_parents(term)
    children = identify_children(term)
    related = identify_related(term)
    add_relationships(term, parents, children, related)
```

Phase 2: 병렬 운영 (2-3주)

기존 시스템	신규 시스템
↓	↓
[고정 계층]	[태그 온톨로지]
↓	↓
운영 지속	테스트 및 검증
↓	↓
점진적 비교 및 보정	

```
# 양쪽 시스템에 동시 저장
def save_question_both_systems(question_text, metadata):
    # 1. 기존 방식
    old_classification = manual_classify(question_text)
    save_to_old_system(old_classification)

    # 2. 새 방식
    tags = auto_tag_question(question_text)
    save_to_new_system(tags)

    # 3. 비교 로그
    log_comparison(old_classification, tags)
```

Phase 3: 과거 데이터 마이그레이션 (1-2주)

```
def migrate_historical_data(batch_size=100):
    """과거 문항 재태깅"""

    total = db.count("SELECT COUNT(*) FROM questions_old")
    batches = (total // batch_size) + 1
```



```

for i in range(batches):
    offset = i * batch_size
    questions = db.execute("""
        SELECT question_id, question_text, category_1, category_2, category_3
        FROM questions_old
        LIMIT %s OFFSET %s
    """, (batch_size, offset)).fetchall()

    for q in questions:
        # 자동 태깅
        tags = auto_tag_question(q['question_text'])

        # 신뢰도 확인
        if tags['confidence'] < 0.7:
            # 기존 분류를 힌트로 사용
            tags = enhance_with_old_classification(
                tags,
                [q['category_1'], q['category_2'], q['category_3']]
            )

        # 새 시스템에 저장
        db.execute("""
            INSERT INTO survey.question_tags (
                question_id, question_text, tags, auto_tagged, tag_confidence
            ) VALUES (%s, %s, %s, %s, %s)
        """, (
            q['question_id'],
            q['question_text'],
            json.dumps(tags['tags']),
            True,
            tags['confidence']
        ))

    print(f"진행률: {(i+1)/batches*100:.1f}%")

```

Phase 4: 검증 및 보정 (1주)

```

# 1. 자동 태깅 정확도 검증
sample = random_sample(migrated_questions, 100)
accuracy = manual_review_accuracy(sample)
print(f"자동 태깅 정확도: {accuracy}%")

# 2. 저신뢰도 문항 수동 보정
low_confidence = db.execute("""
    SELECT * FROM survey.question_tags
    WHERE tag_confidence < 0.7
    ORDER BY tag_confidence ASC
""").fetchall()

for q in low_confidence:

```

```
corrected_tags = manual_correction_ui(q)
update_tags(q['question_id'], corrected_tags)

# 3. 일관성 검증
validate_consistency()
```

Phase 5: 전환 완료 (1주)

- [] 모든 애플리케이션 새 시스템으로 전환
- [] 기존 시스템 읽기 전용으로 전환
- [] 모니터링 및 알림 설정
- [] 팀 교육 및 문서화
- [] 기존 시스템 폐기 일정 수립

6.2 위험 관리

리스크 및 대응

리스크	영향	확률	대응 방안
자동 태깅 정확도 낮음	높음	중간	수동 보정 프로세스, 신뢰도 임계값
온톨로지 설계 오류	높음	낮음	Phase 1에서 충분한 검토, 외부 전문가 자문
성능 저하	중간	낮음	인덱싱, 캐싱, 성능 테스트
팀 저항	중간	중간	충분한 교육, 점진적 전환
데이터 손실	높음	매우낮음	백업, 병렬 운영

롤백 계획

```
# 비상시 기존 시스템으로 즉시 복귀 가능
def rollback_to_old_system():
    # 1. 애플리케이션 라우팅 전환
    update_config(use_system='old')

    # 2. 새 시스템 데이터는 보존
    # (나중에 재시도 가능)

    # 3. 문제 분석 후 재시도
```

7. 실행 계획 (Roadmap)

7.1 타임라인

Week 1-2: 준비 및 온톨로지 구축

- └ 기존 분류 분석
- └ 핵심 용어 추출 (50-100개)
- └ 온톨로지 설계
- └ 관계 설정

Week 3-4: 시스템 개발

- └ DB 스키마 생성
- └ 자동 태깅 엔진 개발
- └ 검색 엔진 개발
- └ 관리 도구 개발

Week 5-6: 테스트 및 검증

- └ 과거 문항 샘플 태깅 (100개)
- └ 정확도 측정 및 개선
- └ 성능 테스트
- └ 병렬 운영 시작

Week 7-8: 마이그레이션

- └ 전체 과거 문항 재태깅
- └ 수동 보정
- └ 일관성 검증
- └ 최종 검토

Week 9: 전환 및 교육

- └ 시스템 전환
- └ 팀 교육
- └ 문서 작성
- └ 모니터링 설정

7.2 성공 지표 (KPI)

정량적 지표

1. 자동 태깅 정확도
목표: 85% 이상
측정: 수동 검토 샘플 100개
2. 태깅 시간
기존: 문항당 5분 (수동 분류)
목표: 문항당 30초 (자동 + 검토)
3. 검색 정확도
목표: Precision 90%, Recall 85%
4. 신규 분류 추가 시간
기존: 5시간/개
목표: 30분/개

5. 시스템 응답 시간
목표: 검색 < 0.5초, 태깅 < 2초

정성적 지표

1. 사용자 만족도
 - 분류 체계 이해도
 - 검색 편의성
 - 시스템 신뢰도
2. 유지보수성
 - 분류 일관성 유지 난이도
 - 온톨로지 관리 편의성
3. 비즈니스 가치
 - 분석 품질 향상
 - 벤치마킹 정확도
 - 인사이트 발견 속도

8. 결론 및 권고사항

8.1 핵심 요약

문제의 본질 설문 문항 분류는 수백~수천 가지 조합이 가능하고 지속적으로 진화하는 복잡한 도메인입니다. 고정된 계층 구조로는 이러한 복잡성과 동적 특성을 수용할 수 없습니다.

제안 솔루션: 하이브리드 접근법 두 가지 계층을 명확히 구분합니다:

1. **구조적 메타데이터** (일반 SQL 컬럼): 진단 유형, 연도, 회사, 업종 등 명확하고 변하지 않는 분류
2. **의미론적 온톨로지** (JSONB 태그): 조직문화, 리더십, 비전 등 복잡하고 진화하는 분류

이를 통해 빠른 필터링과 유연한 의미 검색을 동시에 확보합니다.

핵심 이점

1. **명확한 구조** - 진단 유형/연도/회사는 별도 컬럼으로 빠르게 필터링
2. **무한 확장성** - 새로운 의미론적 개념 즉시 추가 (태그)
3. **다차원 분류** - 한 문항이 여러 의미 차원에 동시 속함
4. **의미 기반 검색** - 관련 개념 자동 검색 (온톨로지)
5. **하이브리드 쿼리** - 구조적 필터 + 의미 검색 조합 가능
6. **장기 비용 절감** - 연간 5주 개발 시간 절약

8.2 권고사항

즉시 실행 (This Week)

1. **의사결정**: 기술팀 + 운영팀 회의로 접근법 합의
2. **샘플링**: 과거 문항 500개 샘플 추출

3. 용어 목록: 자주 등장하는 분류 용어 50개 추출

단기 (1-2 Month)

1. **온톨로지 구축**: 핵심 50-100개 용어와 관계 설정
2. **MVP 개발**: 자동 태깅 + 검색 기본 기능
3. **파일럿 테스트**: 최근 프로젝트 1-2개로 검증

중기 (3-6 Month)

1. **전체 마이그레이션**: 과거 데이터 재태깅
2. **시스템 전환**: 모든 신규 문항 새 시스템 사용
3. **팀 교육**: 온톨로지 개념 및 사용법 교육

장기 (1 Year+)

1. **지속 개선**: 온톨로지 분기별 리뷰 및 확장
2. **고급 기능**: AI 추천, 자동 벤치마킹 매핑
3. **외부 확장**: 다른 유형의 데이터에도 적용

8.3 대안 고려

만약 즉시 전환이 부담스럽다면, **하이브리드 접근**도 가능합니다:

```
-- 기존 계층 + 태그 병행
CREATE TABLE survey.questions_hybrid (
  question_id VARCHAR(50),

  -- 기존 방식 (하위 호환성)
  category_1 VARCHAR(50),
  category_2 VARCHAR(50),
  category_3 VARCHAR(50),

  -- 새 방식 (확장성)
  tags JSONB,

  -- ...
);
```

이렇게 하면:

- 기존 코드 계속 동작
- 새로운 기능은 tags 활용
- 점진적 전환 가능

8.4 최종 의견

개인적 추천: 태그 기반 온톨로지로 전환하는 것이 장기적으로 훨씬 유리합니다. 초기 6주 투자로 향후 수년간 매년 5주씩 절감할 수 있으며, 시스템 품질과 유지보수성이 크게 향상됩니다.

특히 효단님 회사가 장기적으로 이 시스템을 운영하고 발전시킬 계획이라면, **지금**이 전환하기 가장 좋은 시점입니다. 데이터가 더 쌓이기 전에 구조를 바로잡는 것이 미래의 기술 부채를 줄이는 길입니다.

9. 부록

9.1 용어 정의

- **온톨로지 (Ontology)**: 개념들과 그들 간의 관계를 명시적으로 정의한 체계
- **태그 (Tag)**: 콘텐츠를 설명하는 메타데이터 레이블
- **JSONB**: PostgreSQL의 이진 JSON 데이터 타입
- **GIN 인덱스**: Generalized Inverted Index, JSONB 검색 최적화
- **임베딩 (Embedding)**: 텍스트를 고차원 벡터로 변환한 것
- **시멘틱 검색 (Semantic Search)**: 의미 기반 검색

9.2 참고 자료

학술 자료

- Gene Ontology Project: 생물학 온톨로지 사례
- WordNet: 영어 어휘 온톨로지
- Schema.org: 웹 데이터 온톨로지 표준

기술 문서

- PostgreSQL JSONB: <https://www.postgresql.org/docs/current/datatype-json.html>
- NetworkX: <https://networkx.org/>
- Sentence Transformers: <https://www.sbert.net/>

9.3 FAQ

Q: 기존 분류 코드(OD-VIS-UND-v1)는 어떻게 되나요? A: 계속 사용 가능합니다. 태그 시스템과 병행할 수 있습니다. 코드는 사람이 읽기 쉬운 ID 역할을 하고, 태그는 검색과 분석에 사용됩니다.

Q: 자동 태깅이 틀리면 어떻게 하나요? A: 신뢰도 점수(confidence)를 제공하며, 낮은 경우 수동 검토를 거칩니다. 수동 보정한 데이터는 시스템 학습에 활용됩니다.

Q: 온톨로지 관리가 어렵지 않나요? A: 초기 구축 후에는 월 2시간 정도로 관리 가능합니다. 자동 제안 기능이 대부분 처리합니다.

Q: 성능 문제는 없나요? A: PostgreSQL JSONB는 매우 최적화되어 있으며, 적절한 인덱싱으로 수십만 문항도 0.1초 내 검색 가능합니다.

Q: 다른 회사도 이런 방식을 쓰나요? A: Google, Facebook 등 대형 기술 기업들이 콘텐츠 분류에 유사한 태그/온톨로지 기반 시스템을 사용합니다.

Q: 회사 코드와 회사별 특수 태그의 차이는? A:

- `company_code`: 어느 회사의 진단인지 (메타데이터) → 별도 컬럼
- `tags.custom.cj_core_value`: CJ의 핵심가치 "ONLYONE" 개념 포함 여부 (의미론적) → JSONB 태그

Q: 연도와 트렌드의 차이는? A:

- **year**: 몇 년도에 실행된 진단인지 (사실) → 별도 컬럼
- **tags.trends**: "DX", "ESG" 같은 시대적 트렌드 개념 포함 여부 (의미) → JSONB 태그

9.4 실전 예시

예시 1: 2024년 CJ 조직진단 문항

```
INSERT INTO survey.question_tags VALUES (
  'OD-CJ-2024-VIS-001',
  'CJ의 ONLYONE 비전을 명확히 이해하고 있습니까?',

  -- 명확한 메타데이터 (별도 컬럼)
  'OD',          -- diagnosis_type
  2024,          -- year
  'CJ',          -- company_code
  '식품',        -- industry
  'LIKERT_5',    -- scale_type
  FALSE,         -- is_reverse

  -- 의미론적 태그 (JSONB)
  '{
    "themes": ["조직문화", "전략"],
    "concepts": ["비전", "핵심가치"],
    "aspects": ["이해도", "명확성"],
    "subjects": ["조직"],
    "methods": ["자기평가"],
    "custom": {
      "cj_core_value": "ONLYONE"
    }
  }'::jsonb,

  FALSE, -- auto_tagged
  0.95   -- tag_confidence
);
```

예시 2: 2025년 IT업종 디지털전환 리더십 문항

```
INSERT INTO survey.question_tags VALUES (
  'LD-IT-2025-DX-001',
  '리더가 팀원의 AI 활용 역량을 코칭하고 있습니까?',

  -- 명확한 메타데이터
  'LD',          -- diagnosis_type (리더십 진단)
  2025,          -- year
  'TECH001',     -- company_code
  'IT',          -- industry
  'LIKERT_7',    -- scale_type
  FALSE,         -- is_reverse
```

```
-- 의미론적 태그
'{
  "themes": ["리더십", "디지털전환"],
  "concepts": ["코칭", "역량개발", "AI활용"],
  "aspects": ["실행력", "빈도"],
  "subjects": ["리더", "팀원"],
  "methods": ["타인평가"],
  "trends": ["AI", "DX"]
}'::jsonb,

TRUE,    -- auto_tagged
0.88     -- tag_confidence
);
```

예시 3: 쿼리 비교

잘못된 방식 (모두 태그):

```
-- ✗ 나쁜 예: 연도도 태그에 넣기
tags: {
  "year": 2024, -- 이걸 별도 컬럼으로 해야 함!
  "concepts": ["비전"]
}
```

올바른 방식 (하이브리드):

```
-- ☑ 좋은 예: 명확한 것은 컬럼, 의미론적인 것은 태그
WHERE year = 2024 -- 컬럼 필터 (빠름)
AND tags @> '{"concepts": ["비전"]}' -- 태그 검색 (유연함)
```

문서 종료

문의사항이나 추가 논의가 필요하신 경우 언제든지 연락 주시기 바랍니다.

핵심 원칙 재정리:

- 진단 유형, 연도, 회사 → **별도 컬럼** (명확한 메타데이터)
- 조직문화, 리더십, 비전 → **JSONB 태그** (의미론적 온톨로지)
- 몰입도 척도, 리더십 지수 → **온톨로지 관계** (HAS_COMPONENT)
- 두 계층을 혼동하지 말 것!

10. 방법론적 결정이 필요한 이슈들

본 섹션은 시스템 구현 전에 방법론적으로 논의하고 결정해야 할 사항들입니다.

Issue #1: 온톨로지 관계 타입 정의

배경: 척도-컴포넌트 관계를 정의할 때 relation_type을 어떻게 명명할 것인가?

옵션:

1. HAS_COMPONENT

- "몰입도는 자긍심을 컴포넌트로 가진다"
- 장점: 명확함, 기술적
- 단점: 다소 추상적

2. MEASURED_BY

- "몰입도는 자긍심으로 측정된다"
- 장점: 측정 관점에서 직관적
- 단점: 방향성 혼동 가능 (몰입도→자긍심 vs 자긍심→몰입도)

3. CONSISTS_OF

- "몰입도는 자긍심으로 구성된다"
- 장점: 자연스러운 표현
- 단점: COMPONENT와 의미 중복

4. CALCULATED_FROM

- "몰입도는 자긍심으로부터 계산된다"
- 장점: 계산 관점 명확
- 단점: "계산"이라는 용어가 기술적 구현에 치우침

결정 필요 사항:

- 용어 선택 및 표준화
- 다른 기존 relation_type (PARENT, CHILD, RELATED)과의 일관성
- 역관계(inverse relation) 필요 여부

Issue #2: 척도 계산 가중치 관리

배경: 척도 계산 시 각 컴포넌트의 가중치를 어디에 저장하고 관리할 것인가?

시나리오:

몰입도 점수 = (자긍심 × 0.3) + (소속감 × 0.25) + (충성도 × 0.25) + (의무감 × 0.2)

옵션:

Option 1: taxonomy_relations.metadata JSONB

```
INSERT INTO taxonomy_relations VALUES
('몰입도', '자긍심', 'HAS_COMPONENT', 1.0, '{"weight": 0.3}'::jsonb);
```

- 장점: 관계와 가중치를 함께 관리
- 단점: JSONB 쿼리 복잡도 증가

Option 2: 별도 테이블 (scale_definitions)

```
CREATE TABLE scale_definitions (  
  scale_name VARCHAR(100),  
  component_name VARCHAR(100),  
  weight FLOAT,  
  valid_from INT,  
  valid_until INT  
);
```

- 장점: 명확한 구조, 시간 버전 관리 쉬움
- 단점: 테이블 추가, taxonomy_relations와 중복

Option 3: 애플리케이션 레이어

```
# config.py  
SCALE_WEIGHTS = {  
  "몰입도": {  
    "자극심": 0.3,  
    "소속감": 0.25,  
    ...  
  }  
}
```

- 장점: 빠른 변경, 코드 리뷰 가능
- 단점: DB와 분리, 버전 히스토리 관리 어려움

결정 필요 사항:

- 저장 위치
- 가중치 변경 시 기존 계산 결과에 대한 영향도
- 감사(audit) 요구사항

Issue #3: 시간에 따른 척도 정의 변화 추적

배경: 척도를 구성하는 컴포넌트가 시간에 따라 변경될 때 히스토리를 어떻게 관리할 것인가?

시나리오:

```
2023년 몰입도 = {자극심(0.3), 소속감(0.3), 충성도(0.4)}  
2024년 몰입도 = {자극심(0.25), 소속감(0.25), 충성도(0.25), 심리적안전(0.25)}
```

요구사항:

- 과거 시점의 점수를 재계산할 수 있어야 함
- 2023년과 2024년 점수를 비교할 때 정의 변화를 고려해야 함

옵션:

Option 1: relation에 유효기간

```
ALTER TABLE taxonomy_relations ADD COLUMN valid_from INT;
ALTER TABLE taxonomy_relations ADD COLUMN valid_until INT;

-- 2023년 정의
INSERT VALUES ('몰입도', '자극심', 'HAS_COMPONENT', 1.0,
               '{"weight": 0.3}', 2023, 2023);

-- 2024년 정의 (가중치 변경)
INSERT VALUES ('몰입도', '자극심', 'HAS_COMPONENT', 1.0,
               '{"weight": 0.25}', 2024, NULL);
```

- 장점: 같은 테이블에서 시간 추적
- 단점: 쿼리 복잡도 증가

Option 2: 버전 관리 테이블

```
CREATE TABLE scale_versions (
  scale_name VARCHAR(100),
  version INT,
  year INT,
  definition JSONB -- 전체 컴포넌트와 가중치
);
```

- 장점: 버전 단위로 명확한 스냅샷
- 단점: 온톨로지와 분리

Option 3: taxonomy_evolution 테이블 활용

```
-- 기존 evolution 테이블에 change_type='REWEIGHTED' 추가
INSERT INTO taxonomy_evolution VALUES
  ('몰입도', 'REWEIGHTED',
   '{"components": {"자극심": 0.3}}',
   '{"components": {"자극심": 0.25}}',
   '2024년 가중치 조정');
```

- 장점: 변경 이력을 명시적으로 추적
- 단점: 특정 시점 정의 재구성이 복잡

결정 필요 사항:

- 시간 버전 관리 방식
- 과거 시점 재현 요구사항 수준
- 연도별 비교 분석 방법론

Issue #4: 역코딩(Reverse Scoring) 처리

배경: 부정 문항("나는 이직하고 싶다")의 역코딩을 어느 계층에서 처리할 것인가?

문항 예시:

정방향: "우리 회사에서 일하는 것이 자랑스럽다" (5점 = 매우 그렇다)
 역방향: "나는 이 회사를 떠나고 싶다" (5점 = 매우 그렇다 → 역코딩 필요)

옵션:

Option 1: 구조적 메타데이터 (컬럼)

```
ALTER TABLE question_tags ADD COLUMN is_reverse_scored BOOLEAN DEFAULT FALSE;
```

- 논리: 역코딩은 사실(fact)이다. "이 문항은 역코딩이 필요하다"
- 장점: 명확하고 빠른 필터링
- 단점: 또 다른 고정 컬럼 추가

Option 2: 의미론적 태그

```
{
  "concepts": ["이직의도"],
  "aspects": ["역문항"], // 또는
  "scoring": {"reverse": true}
}
```

- 논리: 역코딩은 의미론적 특성이다
- 장점: 태그 체계 내에서 일관성
- 단점: 계산 로직과 의미를 혼동

Option 3: 온톨로지 관계 메타데이터

```
-- "이직의도"는 "몰입도"와 역관계
INSERT INTO taxonomy_relations VALUES
  ('몰입도', '이직의도', 'HAS_COMPONENT', -1.0); -- 음수 강도 = 역관계
```

- 논리: 역관계는 개념 간의 관계
- 장점: 의미론적으로 정확

- 단점: 문항 레벨이 아닌 개념 레벨에서만 정의

결정 필요 사항:

- 역코딩의 본질: 사실인가, 의미인가, 관계인가?
- 문항 레벨 vs 개념 레벨 정의
- 계산 로직과의 연계

Issue #5: 온톨로지 관계의 강도(Strength) 활용

배경: 현재 `taxonomy_relations.strength` FLOAT 컬럼이 있지만 용도가 불명확합니다.

가능한 용도:

Option 1: 가중치로 활용

```
('몰입도', '자극심', 'HAS_COMPONENT', 0.3) -- strength = weight
```

- 장점: 기존 컬럼 활용
- 단점: strength의 의미가 다른 relation_type마다 다름

Option 2: 관계의 확신도

```
('리더십', '코칭', 'RELATED', 0.9) -- 90% 확신  
( '리더십', '권한위임', 'RELATED', 0.7) -- 70% 확신
```

- 장점: 자동 태깅 시 유용
- 단점: 척도 계산 가중치와 별도 필드 필요

Option 3: 사용 안 함 (항상 1.0)

- 가중치는 metadata JSONB 사용
- strength는 reserved for future use

결정 필요 사항:

- strength 컬럼의 정확한 의미 정의
- 가중치와의 관계
- 기본값 규칙

Issue #6: 다중 척도 소속 처리

배경: 하나의 개념이 여러 척도에 동시에 속할 수 있습니다.

시나리오:

"소속감" 개념:

- 몰입도 척도의 컴포넌트 (가중치 0.25)
- 조직문화지수의 컴포넌트 (가중치 0.15)
- 팀워크 점수의 컴포넌트 (가중치 0.3)

문제: 같은 "소속감" 문항들이 여러 척도 계산에 사용됨. 이것이 의도된 것인가?

옵션:

Option 1: 허용 (현재 설계)

- 하나의 개념이 여러 척도에 기여 가능
- 온톨로지에서 여러 관계 정의
- 장점: 유연함
- 단점: 척도 간 독립성 저하

Option 2: 제한 (세부 개념 분리)

"소속감" → 너무 일반적
"조직소속감" → 몰입도용
"팀소속감" → 팀워크용

- 장점: 척도 간 명확한 분리
- 단점: 개념 증식, 온톨로지 복잡도 증가

결정 필요 사항:

- 다중 척도 소속 허용 여부
- 허용 시 가중치 설정 원칙
- 척도 간 상관관계 관리

결론

이상의 이슈들은 **기술 구현 전에 도메인 전문가와 논의하여 방법론적으로 결정**되어야 합니다.

각 이슈에 대한 결정은:

1. 비즈니스 요구사항
2. 데이터 특성
3. 분석 패턴
4. 유지보수 편의성

을 종합적으로 고려하여 이루어져야 합니다.